1. The final grader tarball has been published (khellman2026-03-26T08:01:23-0700), grad student should update.
    - Ugh, `CHECK` mode data populated in tarball now. -- khellman 2026-03-27 12:48:29.
    - Gah, another fix. -- khellman 2026-03-27 13:25:01
    - gradescope slots set up -- khellman 2026-03-27 13:53:23
2. There are two different runtime modes for `WRECK`. Students enrolled in the course for undergraduate credit need to **complete only the REC** runtime mode. Students enrolled for graduate credit should develop logic for **both REC and CHECK runtime modes.**
3. You can also subscribe to this page and receive Emails when changes are made.
4. Here is the change log for this assignment write-up. I will try to be descriptive in my log messages.

In this assignment will you will *close the loop* on our scanner (lexer) implementation. Your application will compile regular expressions into an NFA and write an input `scan.u` file compatible with LUTHOR. The name of your application must be `WRECK` (case sensitive).

`WRECK` will take a command line argument that will be **very similar** to the input for LUTHOR. `WRECK` will produce a `scan.u` file as output as well as several `.nfa` files that can be processed by your NFAMATCH.

Input

Argument	Description	Example Value
1	Runtime mode	`REC` or `CHECK`
2	Context free grammar file	`llre.cfg`
if argument 1 is `REC`		
3	(Input) lexing config file	`scan.lut`
4	(Output) scanner config file	`scan.u`
if argument 1 is `CHECK`		
3	(Input) file of zero or more white space delimited terminals	`input.dat`
4	Count depth parameter	`3`

REC Mode

> All enrolled students' `WRECK` submissions should meet the `REC` runtime mode requirements.

The format of `scan.lut` is identical to `scan.u` (defined in LUTHOR) **except that** in the second and subsequent lines the first field is a **regular expression** instead of an optimized transition table definition file.

> In `REC` mode (**r**egular **e**xpression **c**ompiler) the second argument will always be llre.cfg, our grammar for the course's regular expression notation. This is for the sake of consistency with `CHECK` mode. You may either perform grammar analysis on the grammar, constructing an LL(1) parsing table on each invocation or simply hard-code both the grammar and parsing table into your `WRECK`.

REC Coding

`WRECK`'s first job is to choose a λ character[1] that does not exist in the `scan.lut` alphabet contained on the first line of the file. Since we are using **alphabet encoding**, this doesn't even have to be a printable character.

`WRECK`'s biggest job is to convert regular expressions to NFAs, and then writing those NFAs to disk. Much of this logic has been implemented in your LGA group code base. You are **allowed to use your shared LGA code in this assignment**. Here are the steps your code should take for each regular expression and token id in `scan.lut`.

1. Using llre.cfg, your group's top-down LL(1) parser routines and the "silly lexer" written in lga-re-sdt.pdf, parse each regular expression.

   > - `WRECK` should have an exit status of 4 if a **lexical error** is detected: the regular expression uses a character **not in the RE alphabet** (defined in the first line of `scan.lut`). If the RE $\Sigma = \{a, e, i, o, u\}$, the range operator of our RE language `a-e` would be equivalent to `(a|e)` and **should not** illicit a lexical error. However `b-i` **should be considered a lexical error** because the range end points must be in the RE Σ.
>
> Lexical error testing can be done during token scanning, parsing, or NFA generation; it is your choice where and how it is done --- but it needs to be done.
>
> - `WRECK` should have an exit status of 2 if a **syntax error** is encountered for any regular expression in `scan.lut`.
>
> The final state of `scan.u` and NFA files is ignored in these cases.

2. Using either the completed *concrete syntax tree* (the "raw" parse tree) or by implementing *syntax directed translation* as the parse tree is constructed, derive an RE expression[2] tree as described in lga-re-sdt.pdf. This **abstract syntax tree** (AST) will have internal nodes for only `ALT`, `SEQ`, `*` and (possibly) `+` and `range`; it will have leaf nodes that are either character values, λ leaves, or a `.` (dot) leaf. **It's the student's choice to use + nodes (or convert them to `SEQ` and * subexpressions) and `range` nodes (or convert them to `ALT` subexpressions).**
3. Using the pseudo code solutions of lga-re-nfagen.pdf, convert the RE AST into a λ square matrix L and a transition table T.

    - `WRECK` should have an exit status of 3 if a **semantic error** is encountered for any regular expression in `scan.lut`; the final state of `scan.u` and NFA files is ignored in this case.

    The only possible semantic error[3] is the *dash* range operator with inverted (descending ASCII code) arguments. eg: `z-a` or `)-(`.

4. Write the NFA defined by L and T and your chosen λ character to disk using the NFA file format of NFAMATCH. Recall that while the alphabet line of `scan.u` files may have character values globbed together, the **NFA format requires white space delimited values** everywhere.

REC Output

`WRECK REC ...` will generate two forms of output, the `scan.u` file and NFA files.

scan.u

`WRECK` will store into `scan.u` (the output filename provided in the second command line argument) the data from `scan.lut` **except** that instead of the regular expression of the first field, it will store `tokenid.tt`. Where `tokenid` is the second field of the same line.

For example, if the `scan.lut` file contains

```
x0ax20x5C abcde
(\s|\\|b|c|d)*a     lasta
(\s|a|c|d)*b        lastb
(\s|\\|a|b|d)*c     lastc
d+                  dee         D
.                   IGNORE
```

Then the generated `scan.u` file would be

```
x0a x20 x5C x61x62x63x64x65
lasta.tt            lasta
lastb.tt            lastb
lastc.tt            lastc
dee.tt              dee         D
IGNORE.tt           IGNORE
```

> I've lined the columns up nicely to make this write up easier to read, you don't have to be nearly as organized in your output: just use white space delimited fields.
>
> Also, note that while the first line has been formatted differently, it contains **the same information** as the original `scan.lut` and it's formatting is valid according to the definition in LUTHOR. This is perfectly acceptable.

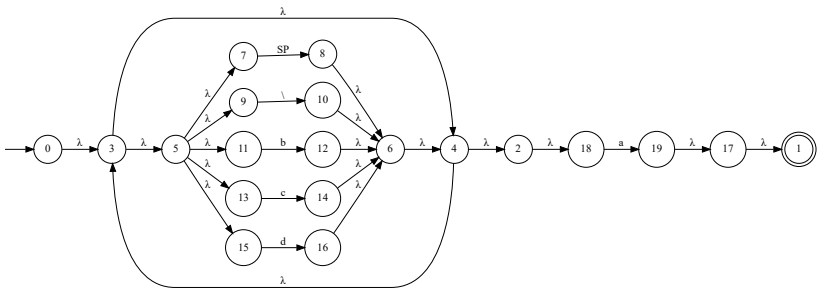
tokenid.nfa

In the example above, the `WRECK` run would have also generated an NFA definition file for each regular expression. **The NFA file must be named `tokenid.nfa` to match the `tokenid.tt` value written in `scan.u`.**

For instance, the contents of `lasta.nfa` **might be**

`lasta.nfa`	NFA visualization

```
20 Z x0a x20
x5c a b c d e
- 7 8  x20
- 9 10  x5c
- 11 12  b
- 13 14  c
- 15 16  d
- 18 19  a
- 6 4 Z
- 5 9 Z
- 17 1 Z
- 10 6 Z
- 16 6 Z
- 19 17 Z
- 5 11 Z
- 5 7 Z
- 12 6 Z
- 5 13 Z
- 5 15 Z
- 2 18 Z
- 4 3 Z
- 4 2 Z
- 8 6 Z
- 0 3 Z
- 14 6 Z
- 3 4 Z
- 3 5 Z
+ 1 1
```



"**Might**" because the choice of λ is near arbitrary, and since this is an **NFA** the number of representations is unbounded.

The format for the NFA file is the same as in NFAMATCH.

> The **tokenid.nfa** file(s) should be written in the `WRECK` process's invocation directory (its *current working directory*).

CHECK Mode

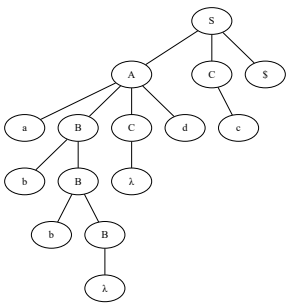
> `WRECK` submissions for all students enrolled for **graduate credit** should meet the `CHECK` runtime mode requirements.

The second command line argument will be a context free grammar file (the same format as used for learning group code, lga-cfg-code.pdf). This will be a consistent grammar (all non-terminals will derive to either λ or a sequence of terminals, all non-terminals[4] will exist in at least one sentential form of the language). All **starting goal** rules will terminate with $ and $ will be in no other production rules.

The fourth command line argument is a non-negative integer value, it will be used in `CHECK` mode **only if the grammar and candidate source do not require a non-zero exist status from `WRECK`** (more details below).

CHECK Code

1. First, the context free grammar should be read and analyzed, culminating in an **LLT**, a LL(1) parsing table for the grammar.
   If an LLT cannot be constructed due to **predict set conflicts**, `WRECK` should `exit(5)` and emit no `OUTPUT` lines.
2. Secondly, the third parameter `input.dat` file should be parsed according to the provided grammar. The input file will contain zero or more white space separated terminals from the grammar file. These may be spread across multiple lines and any line may contain zero or more terminals. If a **syntax error** is encountered `WRECK` should `exit(2)` as in the `REC` runtime mode and not emit any `OUTPUT` lines.
3. Lastly, if the terminals are a valid sentence of the language described by the grammar file, your LL(1) parsing logic should have generated a **concrete syntax tree** (a "raw" parse tree) complete with $ leaf and λ leaves (if a "λ rule" was used in parsing).
   In this case, `WRECK` should consider the fourth parameter (which we'll call `COUNTDEPTH`) and report the number of immediate children belonging to all nodes that are `COUNTDEPTH` edges away from the root of the parse tree.
   - Separate multiple values on one `OUTPUT` line with white space.
   - Leaf nodes should report zero children (of course).
   - Values should be reported from left to right across the appropriate depth of the parse tree.
   - If there is no node at `COUNTDEPTH`, report the string value `NONE`.
   - **An example:** if the grammar and input provided on the command line as arguments two and three of a `CHECK` mode invocation produced the parse tree at the left, the expected output (in various formats, all acceptable and correct) for several `COUNTDEPTH` values are given at the right.



COUNTDEPTH of 1
OUTPUT 4 OUTPUT 1 0
COUNTDEPTH of 2
OUTPUT 0 OUTPUT 2 OUTPUT 1 OUTPUT 0 OUTPUT 0
COUNTDEPTH of 33
OUTPUT :NONE:
COUNTDEPTH of 0
OUTPUT 3

In this last case of a successful parse, as in `REC` mode, `WRECK` should `exit(0)`.

grader.sh

I am providing to students the same grader script that will be used on gradescope for testing your `WRECK`. This singular tarball has both `REC` and `CHECK` mode tests in it. Here is how to use it:

First, fire up your usual course development environment and download this tarball to your development environment and unroll it in a temporary directory.

```
$ ls wreck-student.*
wreck-student.tar.bz2
$ mkdir ~/tmp
$ cd ~/tmp
$ tar xjf ../wreck-student.tar.bz2
:
$ ls
wreck/
```

Now go to the directory holding your `WRECK` and execute the `grader.sh` script from the `wreck-student.tar.bz2` resource.

```
$ cd ~/compilers/WreckerBall
$ ls WRECK
WRECK
$ ~/tmp/wreck/grader.sh
:
:
:
```

If you are enrolled for graduate credit, set the `GRADERMODE` environmental variable to `ALL` before running the grader script. This will perform `CHECK` mode tests after the `REC` mode tests. The default (if `GRADERMODE` is not found in the environment or `GRADERMODE=REC`) is to only run the `REC` mode tests. For example:

```
$ GRADERMODE=ALL ~/tmp/wreck/grader.sh
```

Or, to just run tests specific to `CHECK` mode:

```
$ GRADERMODE=CHECK ~/tmp/wreck/grader.sh
```

You will need to read any messages from the script carefully, and you may need to hit `ENTER` several times throughout its course. This script checks for:

- `REC` Mode
    a. basic i/o errors
    b. lexical errors
    c. semantic errors
    d. syntax errors
    e. and the difference between your `WRECK REC ...` results and expected results.
- `CHECK` Mode
    a. basic i/o errors
    b. LLT conflicts
    c. syntax errors
    d. and the difference between your `WRECK CHECK ...` results and expected results.

All test results are displayed on the terminal screen, but they whip by pretty quickly so you may need to scroll up and make sure you see them all.

When there is a discrepancy in expected results the script either:

- points you to a data file or directory along side your `WRECK` where more failure details are available,
- or all the details are displayed on screen.

Before the `grader.sh` terminates, it will show a summary of some specific rubric line results for the programming project.

Submit Your Work

Partners

You may complete this assignment in a group of two students **that are both rostered for the same course (undergraduate or graduate credit)**.

Checking and Submitting Your Work

Here are some things to double check your submission against:

i. Your archive does not raise errors when "unrolling" with the `${COMPGRADING}/explode-subm` script.

```
$ echo $COMPGRADING
/COMPGRADING
$ ls
my-project.zip
$ "${COMPGRADING}/explode-subm" -P PROJECT my-project.zip ./here
```

ii. `PROJECT` is different for each project, the first two (for example) are `ALPHABETENCODING` and `LUTHOR`.
iii. Your archive contains only the essential files, **don't provide unneeded files**
iv. Don't provide files that have been provided to you (such as test cases in the `*-student.tar.bz2` files); that would just be silly.
v. Make sure the `grader.sh` script for the assignment runs to completion.

> You aren't required to use `.zip` files, `explode-subm` knows about the usual suspects: `.tar`, `.tar.gz`, `.tar.bz2` ...

When you are happy with your work, submit your project archive file for grading at https://www.gradescope.com.

Rubric

This work is worth 65 points for students enrolled in the undergrad course and 90 points for students enrolled in the graduate course.

Requirements	Points	Notes
Meets compilers course project requirements	10	
exit status 1 on IO errors: *inassessible, unwritable or empty files*	10	
(REC) exit status 4 on lexical errors	5	
(REC) exit status 3 on semantic errors	5	
(REC) exit status 2 on syntax errors	5	
(REC) generates correct `scan.u`	10	
(REC) generates correct `tokenid.nfa` NFA files	20	
(CHECK) exit status 5 on LLT conflicts	10	
(CHECK) exit status 2 on syntax errors	5	
(CHECK) `COUNTDEPTH` output	10	

1. Be sure to not choose `NUL` 0x00. (1)
2. AKA *abstract syntax tree*. (2)

3. Can you think of another? If so, let me know! (3)
4. Well, except the *starting goal*. Details, details... (4)

Assignments/WRECK (last edited 2026-04-17 22:21:13 by khellman)